

Requirement 4: Why the Observed Regret is Linear and How to Correct the Baseline

OLA Project – Technical Note

Abstract

Requirement 4 correctly implements a non-stationary multiple-campaign auction with a shared budget: the budget plot is coherent, and cumulative cost approaches the total budget B near the end of the horizon. The apparent issue is that cumulative regret can look almost linear. This does not necessarily indicate a failure of Sliding-Window Combinatorial-UCB, CUSUM Combinatorial-UCB, or the Primal-Dual baseline. The main cause is the benchmark currently used in the code: a dynamic realized clairvoyant, or prophet oracle, that knows the realized competing bids round by round. This adds a persistent information advantage. A more meaningful diagnostic baseline is a piecewise expected clairvoyant computed from the stationary distribution inside each interval.

1 Theoretical Setting

The horizon has T rounds, N campaigns, one shared budget B , and average budget

$$\rho = B/T.$$

Requirement 4 is piecewise stationary: the horizon is divided into intervals $I_1, \dots, I_S$. Inside interval I_s , the distribution P_s of the highest competing bids is fixed; across intervals, it changes. Let $m_{i,t}$ be the highest competing bid for campaign i at time t , and let v_i be its value. For campaign i and bid b ,

$$u_t(i, b) = (v_i - b)\mathbf{1}\{b \geq m_{i,t}\}, \quad c_t(i, b) = b\mathbf{1}\{b \geq m_{i,t}\}.$$

For a joint action $a = (b_1, \dots, b_N)$,

$$U_t(a) = \sum_i u_t(i, b_i), \quad C_t(a) = \sum_i c_t(i, b_i).$$

The realized budget constraint is

$$\sum_{t=1}^T C_t(a_t) \leq B,$$

while the stationary LP view uses the per-round expected constraint

$$\mathbb{E}[C_t(a_t)] \leq \rho.$$

The algorithms in Req4 estimate these quantities online and optimize over feasible joint actions subject to shared-budget and conflict-graph constraints.

2 Current Regret Implementation

The current regret uses a dynamic realized clairvoyant. In simplified form:

```
dynamic_oracle = compute_dynamic_clairvoyant_realized(env, budget)
regret = np.cumsum(dynamic_oracle.utilities - learner.utilities)
```

In the local code, this corresponds to a dynamic LP over the entire realized sequence `m_seq`. It knows every $m_{i,t}$ before selecting the oracle action, then compares the learner against the oracle's realized utility. Mathematically,

$$R_T^{\text{prophet}} = \sum_{t=1}^T U_t(a_t^{\text{prophet}}) - \sum_{t=1}^T U_t(a_t),$$

where

$$a_t^{\text{prophet}} = \arg \max_{a \in \mathcal{A}_t(B)} U_t(a)$$

is chosen after observing the realized vector $(m_{1,t}, \dots, m_{N,t})$. This is stronger than knowing the interval distribution P_s : it is an ex-post selector over the random auction outcomes.

3 Why This Creates a Linear Component

Even within a stationary interval, $m_{i,t}$ is freshly drawn each round. A learner, and even a fair distribution-aware clairvoyant, may know P_s but cannot know the realized m_t before bidding. The prophet can condition on m_t . Therefore,

$$\mathbb{E} \left[\max_{a \in \mathcal{A}} U(a; m) \right] \geq \max_{\pi \in \Pi} \mathbb{E}[U(a \sim \pi; m)],$$

with strict inequality whenever different realizations make different actions optimal. Define the interval information premium

$$\Delta_s = \mathbb{E}_{m \sim P_s}[U(a^{\text{prophet}}(m); m)] - \max_{\pi \in \Pi} \mathbb{E}_{m \sim P_s}[U(a \sim \pi; m)].$$

If $\Delta_s > 0$, a learner that has already identified the best feasible expected policy still loses about Δ_s utility per round against the prophet. Hence

$$R_T^{\text{prophet}} = R_T^{\text{learn}} + \sum_{s=1}^S |I_s| \Delta_s \quad (\text{up to budget-coupling effects}).$$

The second term is linear in T . Thus a near-linear regret curve can be a baseline artifact, not proof that the algorithms have linear learning regret.

4 Correct Baseline

The recommended benchmark is a piecewise expected clairvoyant. For each interval I_s , compute

$$\bar{U}_s(a) = \mathbb{E}_{m \sim P_s}[U(a; m)], \quad \bar{C}_s(a) = \mathbb{E}_{m \sim P_s}[C(a; m)].$$

Then solve the shared-budget LP

$$\begin{aligned} \max_{x_{s,a} \geq 0} \quad & \sum_s |I_s| \sum_a x_{s,a} \bar{U}_s(a) \\ \text{s.t.} \quad & \sum_s |I_s| \sum_a x_{s,a} \bar{C}_s(a) \leq B, \\ & \sum_a x_{s,a} = 1, \quad s = 1, \dots, S. \end{aligned}$$

The resulting regret is

$$R_T^{\text{pw-exp}} = \sum_s |I_s| V_s^* - \sum_{t=1}^T U_t(a_t).$$

This oracle may know the changepoints and the distribution of each stationary segment, but not the realized competing bids before bidding. It is therefore a stochastic non-stationary clairvoyant rather than a prophet.

5 Implementation Sketch

The learners do not need to change first. Add a second benchmark and report the realized dynamic oracle only as a prophet upper bound:

```
pw_oracle = compute_pieewise_expected_clairvoyant(
    intervals=env.intervals,
    bid_sets=env.bid_sets,
    values=env.values,
    budget=env.budget,
    conflict_edges=env.conflict_edges,
)
regret = np.cumsum(pw_oracle.expected_per_round - learner.utilities)
```

In the synthetic environment, $\bar{U}_s$ and $\bar{C}_s$ can be computed from the known generator parameters, or by large independent Monte Carlo samples within each interval.

6 Conclusion

The budget plot suggests that Req4's shared-budget mechanism is working: cumulative cost approaches B near T rather than exhausting the budget too early or leaving it largely unused. The near-linear regret is consistent with comparing online algorithms to a prophet that observes realized bids round by round. Replacing the main benchmark with a piecewise expected clairvoyant separates genuine adaptation regret from the artificial linear information premium introduced by the dynamic realized oracle.